

LAB 2 · 8모드 버튼·LCD 통합

VS Code에서 직접 작성·검증한 다음 장비로 확인한다.

작성일 2026. 09. 10. · 이해리

오픈소스 CLI 통합

목차

프로젝트 시작

RTL·TB·XDC 직접 작성

VS Code 시뮬레이션과 수정

구현과 장치 기록

레포트 공통 예습 전체 목차

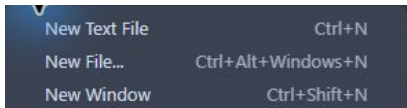
대응 Vivado판

v2.0.1 템플릿을 새 이름으로 clone

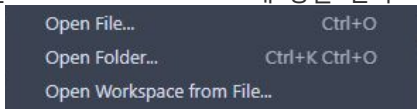
Bash/Zsh/WSL에서 한 줄씩 실행한다.

```
git clone --branch v2.0.1 \  
https://github.com/Glaysia/fpga-lab-template.git \  
lab2_integrated_cli \  
cd lab2_integrated_cli \  
git switch -c main \  
code LAB1.code-workspace
```

File → New Window



Ctrl+Shift+N 또는 File → New Window. 새 창을 연다.



File → Open Workspace from File...을 누른다.

workspace와 추천 확장

1. lab2_integrated_cli 폴더에서 LAB1.code-workspace 선택 → Open.
2. Explorer에 PROJECT 하나와 src.sim.constraints.tools가 보이는지 확인.
3. Extensions → @recommended를 검색.
4. slang.VaporView.vscode-pdf의 Install을 누른다.
5. Python과 Icarus 실행 파일은 별도로 설치되어 있어야 한다.

v2.0.1 편집기와 시뮬레이터

1. slang 확장·서버는 0.3.0을 기준으로 확인한다. 추천 목록은 버전을 고정하지 않는다.
2. 템플릿의 .slang/server.json은 src.sim 탐색용이다. 새 RTL은 simulation.json에도 직접 등록한다.
3. Ctrl+Shift+P → slang: Show Output. 서버 오류가 있으면 README의 버전 확인·재시작 절차를 따른다.
4. slang.path에 다른 사람의 설치 경로를 복사하지 않는다. 직접 지정할 때는 User 설정에 자신의 경로를 쓴다.
5. VS Code 02 Simulate는 Icarus, Vivado Run Behavioral Simulation은 XSim이다.

템플릿 도구 설치·오류 해결 안내

시뮬레이션 도구 확인

1. Terminal → Run Task... → 01 Check tools.
2. Git·Python·iverilog·vvp의 버전이 출력되는지 확인한다.
3. 도구를 찾지 못하면 설치 경로를 PATH에 추가하고 VS Code를 다시 연다.

```
git --version  
python3 --version  
iverilog -V  
vvp -V
```

빈 파일에서 시작

1. src/design.v를 우클릭 → Rename. 첫 RTL 파일명으로 바꾼다.
2. 추가 RTL은 src를 우클릭 → New File로 만든다.
3. sim/tb_design.v를 이번 TB 파일명으로 바꾼다.
4. constraints/pins.xdc를 이번 XDC 파일명으로 바꾼다.
5. 뒤의 코드 화면을 보며 작성하고 File → Save All.

이번 프로젝트에 작성할 파일

역할	경로
src	src/counter4.v
src	src/clock_divider.v
src	src/register_pair.v
src	src/shift_register4.v
src	src/piso4.v

이번 프로젝트에 작성할 파일

역할	경로
src	src/moore_cycle.v
src	src/mealy_toggle.v
src	src/segment_scan8.v
src	src/input_frontend.v
src	src/button_onepulse.v

이번 프로젝트에 작성할 파일

역할	경로
src	src/lcd_lab2_modes.v
src	src/lab2_integrated.v
sim	sim/tb_lab2_integrated.sv
constraints	constraints/lab2_integrated.xdc

simulation.json의 세 항목

`sources`: 앞 표의 `src/` 파일 경로를 배열로 넣는다.

`testbench`: `sim/tb_lab2_integrated.sv`

`simulation_top`: `tb_lab2_integrated`

큰따옴표·대괄호·심표를 확인한다. `module` 이름에는 파일 확장자를 붙이지 않는다.

simulation.json · 1-16행

simulation.json를 열고 해당 구간을 직접 입력한다.

```
1  {
2    "sources": [
3      "src/counter4.v",
4      "src/clock_divider.v",
5      "src/register_pair.v",
6      "src/shift_register4.v",
7      "src/piso4.v",
8      "src/moore_cycle.v",
9      "src/mealy_toggle.v",
10     "src/segment_scan8.v",
11     "src/input_frontend.v",
12     "src/button_onepulse.v",
13     "src/lcd_lab2_modes.v",
14     "src/lab2_integrated.v"
15   ],
16   "testbench": "sim/tb_lab2_integrated.sv",
```

simulation.json · 17-18행

simulation.json를 열고 해당 구간을 직접 입력한다.

```
17 | "simulation_top": "tb_lab2_integrated"  
18 | }
```

이번 회로의 관찰 방법

K4 초기화, N8 모드 변경, N4 회로 한 단계 실행. 새 모드로 넘어갈 때 회로 상태를 초기화한다.

입력 스위치를 먼저 정한 다음 버튼을 누른다.

두 버튼의 역할을 나눈다

입력	역할
B6 / clk	1 kHz 주 클록
K4 / rst	전체 초기화, 첫 모드로 복귀
N8 / mode_button	다음 모드 선택
N4 / step_button	선택한 회로 한 단계 실행
SW1-SW8	sw[7]-sw[0], 모드별 입력

개별 교안의 한 단계 버튼 N8은 통합판에서 N4로 바뀐다. N8은 모드 선택에 사용한다. 스위치를 먼저 정하고 버튼을 누른 뒤 놓는다.

내부 번호와 LCD 번호 · 1-4

mode	LCD 첫 줄	둘째 줄
0	MODE 01	UP DOWN COUNTER
1	MODE 02	CLOCK DIVIDER
2	MODE 03	REGISTER PAIR
3	MODE 04	SHIFT REGISTER

mode는 3비트로 0부터 시작한다. 학생에게 표시하는 LCD 번호는 1부터 시작한다. 두 번호를 레포트에서 구분한다.

내부 번호와 LCD 번호 · 5-8

mode	LCD 첫 줄	둘째 줄
4	MODE 05	PISO
5	MODE 06	MOORE FSM
6	MODE 07	MEALY FSM
7	MODE 08	8 DIGIT SCAN

각 줄은 16칸이며 남는 칸은 공백으로 덮어쓴다. MODE 08에서 N8을 한 번 누르면 MODE 01로 돌아온다.

여덟 회로는 하나의 bit에 함께 들어간다

여덟 코어를 인스턴스화하고 mode에 따라 enable과 LED 출력 선택을 바꾼다. 모드를 바꿀 때마다 bit를 다시 기록하는 구조가 아니다.

counter·register·shift·PISO·Moore·Mealy는 해당 모드의 step_press로 진행한다. 스캔은 mode=7에서 자동으로 진행한다.

분주기는 주 클록으로 계속 동작하며 mode=1에서 출력을 관찰한다. 모든 저장소의 클록은 공통 clk다.

모드를 바꾸면 상태를 초기화한다

circuit_reset = reset || mode_press다. 모드 변경 펄스가 들어온 상승 에지에는 여덟 코어가 초기화되고 mode가 다음 값으로 바뀐다.

같은 에지에서 step_press도 1이어도 코어의 reset이 우선한다. 이전 모드의 마지막 상태를 다음 모드로 넘기지 않는다.

카운터를 15로 만든 뒤 다른 모드를 돌아 다시 MODE 01에 오면 0이다. LCD 회로는 mode_press로 리셋하지 않고 새 표시값을 다음 갱신 주기에 반영한다.

LED 선택과 사용하지 않는 표시 장치

MODE	LED[7:0]
01	0000, count
02	000, tick, /1000, /50, /10, /2
03	stored, registered
04	0000, shifted
05	parallel, 000, serial_out
06	000000, moore_value
07	00000, mealy_state, mealy_value
08	00000, scan_index

스캔 모드 외에는 seg_data=00, seg_com=FF로 7세그먼트를 끈다. LCD는 모든 모드에서 현재 이름을 표시한다.

LCD 한 바이트의 네 단계

phase	동작
0	E=0에서 RS·DATA 준비
1	E=1, 데이터 유지
2	E=0으로 내려 전송
3	회복 시간을 둔 뒤 다음 바이트

1 kHz에서 각 단계는 1 ms다. lcd_rw=0으로 쓰기만 한다. 초기 POWER_WAIT=50은 50클록 대기이며 TB에서도 이 클록 수는 유지한다.

LCD 갱신 중 모드가 바뀌면

첫 줄 주소 80을 전송할 때 mode를 shown_mode에 저장한다. 번호와 둘째 줄 이름은 shown_mode로 계산한다.

따라서 한 화면을 쓰는 동안 버튼을 눌러도 번호와 이름을 서로 다른 모드에서 가져오지 않는다. 다음 화면 갱신에서 최신 mode를 반영한다.

초기화 명령 뒤에는 두 줄의 주소·문자를 반복 전송한다. LED 모드 변화와 LCD 글자 갱신이 완전히 같은 순간일 필요는 없다.

개별 회로 설명으로 이동

카운터 분주 레지스터 시프트

PISO Moore Mealy 7세그먼트

각 코어의 상태 표·TB·수정 실험을 먼저 읽는다. 통합판은 같은 코어를 연결하며 이후 전체 코드 화면을 직접 작성한다.

counter4.v · 1-13행

src/counter4.v를 열고 해당 구간을 직접 입력한다.

```
1 | timescale 1ns/1ps
2 | module counter4 (
3 |     input wire clk, rst, enable, down,
4 |     output reg [3:0] value
5 | );
6 |     always @(posedge clk) begin
7 |         if (rst) value <= 4'd0;
8 |         else if (enable) begin
9 |             if (down) value <= value - 4'd1;
10 |            else value <= value + 4'd1;
11 |         end
12 |     end
13 | endmodule
```

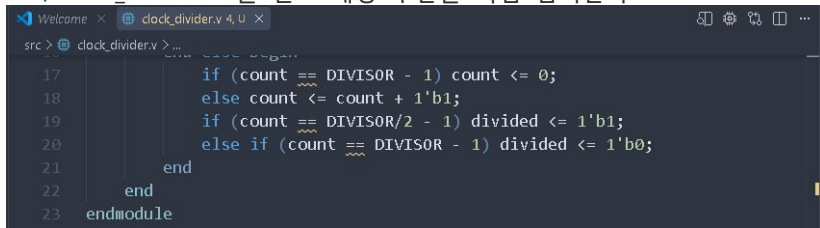
clock_divider.v · 1-16행

src/clock_divider.v를 열고 해당 구간을 직접 입력한다.

```
Welcome x clock_divider.v 4. U x
src > @ clock_divider.v > ...
1 | timescale 1ns/1ps
2 // DIVISOR must be an even integer >= 2.
3 // divided is an observation output; use tick as an enable on clk.
4 module clock_divider #(parameter integer DIVISOR = 10) (
5     input wire clk, rst,
6     output reg divided,
7     output wire tick
8 );
9     localparam integer WIDTH = $clog2(DIVISOR);
10    reg [WIDTH-1:0] count;
11    assign tick = !rst && (count == DIVISOR - 1);
12    always @(posedge clk) begin
13        if (rst) begin
14            count <= 0;
15            divided <= 1'b0;
16        end else begin
```

clock_divider.v · 17-23행

src/clock_divider.v를 열고 해당 구간을 직접 입력한다.



```
src > @ clock_divider.v > ...  
17         if (count == DIVISOR - 1) count <= 0;  
18         else count <= count + 1'b1;  
19         if (count == DIVISOR/2 - 1) divided <= 1'b1;  
20         else if (count == DIVISOR - 1) divided <= 1'b0;  
21     end  
22 end  
23 endmodule
```

register_pair.v · 1-16행

src/register_pair.v를 열고 해당 구간을 직접 입력한다.

```
src > @ register_pair.v > ...
1 | timescale 1ns/1ps
2 | module register_pair (
3 |     input wire clk, rst, load, transfer,
4 |     input wire [3:0] data_in,
5 |     output reg [3:0] stored, value
6 | );
7 |     always @(posedge clk) begin
8 |         if (rst) begin
9 |             stored <= 4'd0;
10 |            value <= 4'd0;
11 |         end else begin
12 |             if (load) stored <= data_in;
13 |             if (transfer) value <= stored;
14 |         end
15 |     end
16 | endmodule
```

shift_register4.v · 1-11행

src/shift_register4.v를 열고 해당 구간을 직접 입력한다.

```
{ } LAB7.code-workspace M × shift_register4.v U ×
src > shift_register4.v > ...
1  timescale 1ns/1ps
2  // Serial input enters bit 3 and moves toward bit 0.
3  module shift_register4 (
4      input wire clk, rst, enable, serial_in,
5      output reg [3:0] value
6  );
7      always @(posedge clk) begin
8          if (rst) value <= 4'd0;
9          else if (enable) value <= {serial_in, value[3:1]};
10     end
11 endmodule
```

piso4.v · 1-14행

src/piso4.v를 열고 해당 구간을 직접 입력한다.

```
1  `timescale 1ns/1ps
2  module piso4 (
3      input wire clk, rst, load, enable,
4      input wire [3:0] data_in,
5      output wire serial_out,
6      output reg [3:0] value
7  );
8      assign serial_out = value[3];
9      always @(posedge clk) begin
10         if (rst) value <= 4'd0;
11         else if (load) value <= data_in;
12         else if (enable) value <= {value[2:0], 1'b0};
13     end
14 endmodule
```

moore_cycle.v · 1-16행

src/moore_cycle.v를 열고 해당 구간을 직접 입력한다.

```
1 `timescale 1ns/1ps
2 module moore_cycle (
3     input wire clk, rst, enable, advance,
4     output reg [1:0] value
5 );
6     always @(posedge clk) begin
7         if (rst) value <= 2'b00;
8         else if (enable && advance) begin
9             case (value)
10                2'b00: value <= 2'b01;
11                2'b01: value <= 2'b10;
12                default: value <= 2'b00;
13            endcase
14        end
15    end
16 endmodule
```

mealy_toggle.v · 1-13행

src/mealy_toggle.v를 열고 해당 구간을 직접 입력한다.

```
1  `timescale 1ns/1ps
2  module mealy_toggle (
3      input wire clk, rst, enable, bit_in,
4      output reg state,
5      output wire [1:0] value
6  );
7      always @(posedge clk) begin
8          if (rst) state <= 1'b0;
9          else if (enable && bit_in) state <= ~state;
10     end
11     // Reset changes state at the rising edge; value is combinational.
12     assign value = !bit_in ? 2'b00 : (state ? 2'b01 : 2'b10);
13 endmodule
```

segment_scan8.v · 1-16행

src/segment_scan8.v를 열고 해당 구간을 직접 입력한다.

```
1 | timescale 1ns/1ps
2 | // Logical outputs are active-high. Apply board polarity in the wrapper.
3 | // Each slot contains one active interval followed by one blank interval.
4 | module segment_scan8 (
5 |     input wire clk, rst, enable,
6 |     input wire [31:0] digits,
7 |     output reg [7:0] select,
8 |     output reg [7:0] segments,
9 |     output reg [2:0] index
10 | );
11 |     reg blank;
12 |     reg [3:0] nibble;
13 |     always @(posedge clk) begin
14 |         if (rst) begin
15 |             index <= 0;
16 |             blank <= 1'b1;
```

segment_scan8.v · 17-32행

src/segment_scan8.v를 열고 해당 구간을 직접 입력한다.

```
17         end else if (enable) begin
18             blank <= ~blank;
19             if (!blank) index <= index + 1'b1;
20         end
21     end
22     always @* begin
23         nibble = digits >> (index * 4);
24         select = blank ? 8'h00 : (8'h01 << index);
25         // abcdefg, dp; decimal point remains off.
26         case (nibble)
27             4'h0: segments = 8'b11111100;
28             4'h1: segments = 8'b01100000;
29             4'h2: segments = 8'b11011010;
30             4'h3: segments = 8'b11110010;
31             4'h4: segments = 8'b01100110;
32             4'h5: segments = 8'b10110110;
```

segment_scan8.v · 33–46행

src/segment_scan8.v를 열고 해당 구간을 직접 입력한다.

```
33      4'h6: segments = 8'b10111110;  
34      4'h7: segments = 8'b11100000;  
35      4'h8: segments = 8'b11111110;  
36      4'h9: segments = 8'b11110110;  
37      4'ha: segments = 8'b11101110;  
38      4'hb: segments = 8'b00111110;  
39      4'hc: segments = 8'b10011100;  
40      4'hd: segments = 8'b01111010;  
41      4'he: segments = 8'b10011110;  
42      4'hf: segments = 8'b10001110;  
43      default: segments = 8'h00;  
44  endcase  
45  end  
46 endmodule
```

input_frontend.v · 1-16행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
1  timescale 1ns/1ps
2  // Main clock is 1 kHz: STABLE_CYCLES=20 means 20 ms.
3  // Set switches first, then press the button; the bus is not an atomic CDC.
4  module input_frontend #(parameter integer STABLE_CYCLES = 20) (
5      input wire clk, rst, button,
6      input wire [7:0] sw,
7      output wire reset,
8      output reg press,
9      output wire [7:0] switches
10 );
11     (* ASYNC_REG = "TRUE" *) reg [1:0] reset_pipe;
12     (* ASYNC_REG = "TRUE" *) reg button_meta, button_sync;
13     (* ASYNC_REG = "TRUE" *) reg [7:0] sw_meta, sw_sync;
14     localparam integer WIDTH = (STABLE_CYCLES < 2) ? 1 : $clog2(STABLE_CYCLES);
15     reg [WIDTH-1:0] count;
16     reg accepted;
```

input_frontend.v · 17-32행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
17  assign reset = reset_pipe[1];
18  assign switches = sw_sync;
19  // Asynchronous assertion; release follows two main-clock edges.
20  always @(posedge clk or posedge rst) begin
21      if (rst) reset_pipe <= 2'b11;
22      else reset_pipe <= {reset_pipe[0], 1'b0};
23  end
24  always @(posedge clk or posedge reset) begin
25      if (reset) begin
26          button_meta <= 0; button_sync <= 0;
27          sw_meta <= 0; sw_sync <= 0;
28      end else begin
29          button_meta <= button; button_sync <= button_meta;
30          sw_meta <= sw; sw_sync <= sw_meta;
31      end
32  end
```

input_frontend.v · 33–46행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
33     always @(posedge clk or posedge reset) begin
34         if (reset) begin
35             count <= 0; accepted <= 0; press <= 0;
36         end else begin
37             press <= 0;
38             if (button_sync == accepted) count <= 0;
39             else if (count == STABLE_CYCLES - 1) begin
40                 accepted <= button_sync;
41                 count <= 0;
42                 press <= button_sync;
43             end else count <= count + 1'b1;
44         end
45     end
46 endmodule
```

button_onepulse.v · 1-16행

src/button_onepulse.v를 열고 해당 구간을 직접 입력한다.

```
1 | timescale 1ns/1ps
2 | module button_onepulse #(parameter integer STABLE_CYCLES=20)(
3 |   input wire clk,rst,button, output reg pulse);
4 |   (* ASYNC_REG="TRUE" *) reg sync1, sync2;
5 |   reg accepted;
6 |   integer count;
7 |   always @(posedge clk or posedge rst) begin
8 |     if(rst) begin sync1<=0; sync2<=0; end
9 |     else begin sync1<=button; sync2<=sync1; end
10 |   end
11 |   always @(posedge clk or posedge rst) begin
12 |     if(rst) begin count<=0; accepted<=0; pulse<=0; end
13 |     else begin
14 |       pulse<=0;
15 |       if(sync2==accepted) count<=0;
16 |       else if(count==STABLE_CYCLES-1) begin
```

button_onepulse.v · 17-21행

src/button_onepulse.v를 열고 해당 구간을 직접 입력한다.

```
17         accepted<=sync2; count<=0; pulse<=sync2;
18         end else count<=count+1;
19     end
20 end
21 endmodule
```

lcd_lab2_modes.v · 1-16행

src/lcd_lab2_modes.v를 열고 해당 구간을 직접 입력한다.

```
1 | timescale 1ns/1ps
2 | // Board source: 2022 CHARACTER LCD guide p13/p30, 1 kHz clock, 16x2, 8-bit bus.
3 | // Each byte has 1 ms setup, 1 ms enable-high and >=2 ms recovery.
4 | module lcd_lab2_modes #(parameter integer POWER_WAIT=50)(
5 |   input wire clk,rst, input wire [3:0] mode,
6 |   output reg lcd_e,lcd_rs, output wire lcd_rw, output reg [7:0] lcd_data);
7 |   reg [1:0] phase;
8 |   integer power_count;
9 |   reg [5:0] index;
10 |  reg [3:0] shown_mode;
11 |  reg [127:0] name;
12 |  assign lcd_rw=1'b0;
13 |  always @* begin
14 |    case(shown_mode)
15 |      0:name="UP DOWN COUNTER ";
16 |      1:name="CLOCK DIVIDER  ";
```

lcd_lab2_modes.v • 17-32행

src/lcd_lab2_modes.v를 열고 해당 구간을 직접 입력한다.

```
17 2:name="REGISTER PAIR ";
18 3:name="SHIFT REGISTER ";
19 4:name="PISO ";
20 5:name="MOORE FSM ";
21 6:name="MEALY FSM ";
22 7:name="8 DIGIT SCAN ";
23 default:name="RESET REQUIRED ";
24 endcase
25 end
26 always @(posedge clk or posedge rst) begin
27   if(rst) begin
28     phase<=0; power_count<=0; index<=0; shown_mode<=0;
29     lcd_e<=0; lcd_rs<=0; lcd_data<=0;
30   end else if(power_count<POWER_WAIT) power_count<=power_count+1;
31   else case(phase)
32     0:begin
```

lcd_lab2_modes.v • 33–48행

src/lcd_lab2_modes.v를 열고 해당 구간을 직접 입력한다.

```
33    lcd_e<=0; lcd_rs<=0;
34    case(index)
35        0,1,2:lcd_data<=8'h38;
36        3:lcd_data<=8'h0c;
37        4:lcd_data<=8'h06;
38        5:begin lcd_data<=8'h80; shown_mode<=mode; end
39        6:begin lcd_rs<=1; lcd_data<="M"; end
40        7:begin lcd_rs<=1; lcd_data<="0"; end
41        8:begin lcd_rs<=1; lcd_data<="D"; end
42        9:begin lcd_rs<=1; lcd_data<="E"; end
43        10:begin lcd_rs<=1; lcd_data<=" "; end
44        11:begin lcd_rs<=1; lcd_data<="0"; end
45        12:begin lcd_rs<=1; lcd_data<=("1"+shown_mode); end
46        22:lcd_data<=8'h0c;
47    default:begin
48        lcd_rs<=1;
```

lcd_lab2_modes.v · 49–64행

src/lcd_lab2_modes.v를 열고 해당 구간을 직접 입력한다.

```
49     if(index>23 && index<=38) lcd_data<=name[127-(index-23)*8 -: 8];
50     else lcd_data<=" ";
51     end
52   endcase
53   phase<=1;
54 end
55 1:begin lcd_e<=1; phase<=2; end
56 2:begin lcd_e<=0; phase<=3; end
57 3:begin
58   // First function-set recovery is 6 ms including setup of the next byte.
59   if(index==0 && power_count<POWER_WAIT+2) power_count<=power_count+1;
60   else begin phase<=0; index<=(index==38)?5:index+1; end
61 end
62 endcase
63 end
64 endmodule
```

lab2_integrated.v · 1-16행

src/lab2_integrated.v를 열고 해당 구간을 직접 입력한다.

```
1 | timescale 1ns/1ps
2 | // Main clock: B6 at 1 kHz. K4 reset, N8 mode, M4 step.
3 | module lab2_integrated #(
4 |     parameter integer STABLE_CYCLES = 20,
5 |     parameter integer DIVISOR = 1000,
6 |     parameter integer LCD_POWER_WAIT = 50
7 | ) (
8 |     input wire clk, rst, mode_button, step_button,
9 |     input wire [7:0] sw,
10 |    output reg [7:0] led,
11 |    output wire [7:0] seg_data, seg_com,
12 |    output wire lcd_e, lcd_rs, lcd_rw,
13 |    output wire [7:0] lcd_data
14 | );
15 |    wire reset, mode_press, step_press;
16 |    wire [7:0] switches;
```

lab2_integrated.v · 17-32행

src/lab2_integrated.v를 열고 해당 구간을 직접 입력한다.

```
17 reg [2:0] mode;
18 input_frontend #(.STABLE_CYCLES(STABLE_CYCLES)) inputs(
19     clk, rst, mode_button, sw, reset, mode_press, switches);
20 button_onepulse #(.STABLE_CYCLES(STABLE_CYCLES)) step_input(
21     clk, reset, step_button, step_press);
22 always @(posedge clk) begin
23     if (reset) mode <= 0;
24     else if (mode_press) mode <= mode + 1'b1;
25 end
26 // Each new mode starts from its initial state, even if step is pressed too.
27 wire circuit_reset = reset || mode_press;
28 wire [3:0] count, stored, registered, shifted, parallel;
29 wire divided, tick, div2, div10, div50, serial_out, mealy_state;
30 wire [1:0] moore_value, mealy_value;
31 wire [7:0] selected, segments;
32 wire [2:0] scan_index;
```

lab2_integrated.v • 33–48행

src/lab2_integrated.v를 열고 해당 구간을 직접 입력한다.

```
33 counter4 counter(clk, circuit_reset, step_press && mode==0, switches[0], count);
34 clock_divider #(.DIVISOR(DIVISOR)) divider(clk, circuit_reset, divided, tick);
35 clock_divider #(.DIVISOR(2)) ratio2(.clk(clk), .rst(circuit_reset), .divided(div2), .tick());
36 clock_divider #(.DIVISOR(10)) ratio10(.clk(clk), .rst(circuit_reset), .divided(div10), .tick());
37 clock_divider #(.DIVISOR(50)) ratio50(.clk(clk), .rst(circuit_reset), .divided(div50), .tick());
38 register_pair registers(clk, circuit_reset,
39     step_press && mode==2 && switches[0],
40     step_press && mode==2 && switches[1], switches[7:4], stored, registered);
41 shift_register4 shift(clk, circuit_reset, step_press && mode==3, switches[7], shifted);
42 piso4 piso(clk, circuit_reset, step_press && mode==4 && switches[0],
43     step_press && mode==4 && !switches[0], switches[7:4], serial_out, parallel);
44 moore_cycle moore(clk, circuit_reset, step_press && mode==5, switches[7], moore_value);
45 mealy_toggle mealy(clk, circuit_reset, step_press && mode==6, switches[7], mealy_state, mealy_value);
46 segment_scan8 scan(clk, circuit_reset, mode==7,
47     {28'h7654321, switches[7:4]}, selected, segments, scan_index);
48 assign seg_data = (mode==7) ? segments : 8'h00;
```

lab2_integrated.v · 49–64행

src/lab2_integrated.v를 열고 해당 구간을 직접 입력한다.

```
49 assign seg_com = (mode==7) ?
50     ~{selected[0],selected[1],selected[2],selected[3],
51       selected[4],selected[5],selected[6],selected[7]} : 8'hff;
52 always @* begin
53     led = 0;
54     case (mode)
55         0: led = {4'b0, count};
56         1: led = {3'b0, tick, divided, div50, div10, div2};
57         2: led = {stored, registered};
58         3: led = {4'b0, shifted};
59         4: led = {parallel, 3'b0, serial_out};
60         5: led = {6'b0, moore_value};
61         6: led = {5'b0, mealy_state, mealy_value};
62         7: led = {5'b0, scan_index};
63     endcase
64 end
```

lab2_integrated.v · 65–67행

src/lab2_integrated.v를 열고 해당 구간을 직접 입력한다.

```
65     lcd_lab2_modes #(.POWER_WAIT(LCD_POWER_WAIT)) display(  
66         clk, reset, {1'b0,mode}, lcd_e, lcd_rs, lcd_rw, lcd_data);  
67 endmodule
```

통합 TB에서 빠르게 만든 부분

STABLE_CYCLES=3, DIVISOR=10으로 버튼과 분주 대기를 줄인다. TB 클럭 주기는 10 ns다.

실제 보드 기본값은 STABLE_CYCLES=20, DIVISOR=1000이며 클럭은 1 kHz다. TB 수치를 XDC 주거나 실제 분주 속도로 옮기지 않는다.

같은 코어 검사 외에 두 버튼·입력 동기화·모드 선택·LCD 버스·COM극성까지 전체 연결을 검사한다.

LCD 이름 ROM을 읽어 정답으로 쓰지 않는다

TB는 실제 lcd_data·lcd_rs·lcd_e 버스를 감시해 32칸 screen을 만든다. 명령 80과 C0으로 줄 주소를 바꾸고 데이터 바이트를 저장한다.

E 상승 때 write-only와 setup 시간을, E 하강 때 E 유지 시간을 검사한다. E가 높은 동안 DATA 또는 RS가 바뀌면 즉시 실패한다.

check_lcd는 300클록을 기다린 뒤 첫 줄 번호·공백과 둘째 줄 16문자를 별도 기대 문자열에 대조한다.

여덟 모드에서 검사하는 것

모드	통합 검사
01	증가·감소·0에서 15로 순환
02	30클록에 tick 3개, TB /10
03	동시 저장·전달 시 이전 값
04	입력 1,0에 LED 08,04
05	병렬 load와 MSB 직렬 출력
06	입력만 바뀌서는 상태 유지
07	버튼 없이 입력으로 출력 변경
08	COM 극성·입력 9·blank 비율

동시 버튼과 실행 중 리셋

버튼 입력에는 짧은 바운스와 길게 누름을 포함한다. next_mode는 한 동작에 모드가 정확히 한 번만 바뀌는지 검사한다.

MODE 08 → MODE 01 복귀 때 이전 카운터 값이 0으로 초기화되는지 확인한다. 이어 두 버튼을 동시에 눌러 코어 갱신보다 초기화가 우선하는지 확인한다.

마지막 전체 리셋은 mode=0, LED=0, lcd_e=0, seg_com=FF를 검사한다. 기준 로그는 modes=8, checks=2848, 종료 34326 ns다.

tb_lab2_integrated.sv · 1-16행

sim/tb_lab2_integrated.sv를 열고 해당 구간을 직접 입력한다.

```
1  `timescale 1ns/1ps
2  module tb_lab2_integrated;
3      reg clk=0, rst=0, mode_button=0, step_button=0;
4      reg [7:0] sw=0;
5      wire [7:0] led, seg_data, seg_com, lcd_data;
6      wire lcd_e, lcd_rs, lcd_rw;
7      // Accelerated debounce/divider; LCD byte timing keeps the default cycle counts.
8      lab2_integrated #( .STABLE_CYCLES(3), .DIVISOR(10)) dut(
9          clk,rst,mode_button,step_button,sw,led,seg_data,seg_com,
10         lcd_e,lcd_rs,lcd_rw,lcd_data);
11     always #5 clk=~clk;
12     integer checks=0, address=0, i, pulses=0, active_slots=0, blank_slots=0;
13     reg [7:0] screen[0:31];
14     reg [127:0] names[0:7];
15     time last_data_change=0, last_enable_rise=0;
16     reg saw_enable=0;
```

tb_lab2_integrated.sv · 17-32행

sim/tb_lab2_integrated.sv를 열고 해당 구간을 직접 입력한다.

```
17 task step; begin @(posedge clk); #1; end endtask
18 task check(input condition, input [8*100-1:0] description);
19 begin
20     checks=checks+1;
21     if(condition!==1'b1) begin
22         $display("LAB2_INTEGRATED_FAIL %0s time=%0t",description,$time);
23         $fatal(1,"check failed");
24     end
25 end endtask
26 // Decode the actual LCD bus rather than reading the controller's name ROM.
27 always @(lcd_data or lcd_rs) begin
28     if(lcd_e===1'b1) $fatal(1,"LCD data changed while enable high");
29     last_data_change=$time;
30 end
31 always @(posedge lcd_e) begin
32     check(lcd_rw===0,"LCD is write only");
```

tb_lab2_integrated.sv · 33–48행

sim/tb_lab2_integrated.sv를 열고 해당 구간을 직접 입력한다.

```
33     check($time-last_data_change>=10,"LCD setup at least one main clock");
34     last_enable_rise=$time; saw_enable=1;
35 end
36 always @(negedge lcd_e) if(saw_enable) begin
37     check($time-last_enable_rise>=10,"LCD enable high at least one main clock");
38     saw_enable=0;
39     if(lcd_rs) begin
40         if(address<32) screen[address]=lcd_data;
41         address=address+1;
42     end else if(lcd_data==8'h80) address=0;
43     else if(lcd_data==8'hc0) address=16;
44 end
45 task switches(input [7:0] value);
46 begin sw=value; repeat(4) step; end endtask
47 task push_step;
48 begin
```

tb_lab2_integrated.sv · 49–64행

sim/tb_lab2_integrated.sv를 열고 해당 구간을 직접 입력한다.

```
49      step_button=1; step; step_button=0; repeat(3) step;
50      step_button=1; repeat(10) step;
51      step_button=0; repeat(10) step;
52  end endtask
53  task next_mode(input [2:0] expected);
54  begin
55      mode_button=1; step; mode_button=0; repeat(3) step;
56      mode_button=1; repeat(10) step;
57      mode_button=0; repeat(10) step;
58      check(dut.mode===expected,"mode advances exactly once despite bounce");
59      if(expected!=7) check(seg_com===8'hff && seg_data===0,"array off outside scan mode");
60  end endtask
61  task check_lcd(input integer mode_number);
62  integer c;
63  begin
64      repeat(300) step;
```

tb_lab2_integrated.sv · 65–80행

sim/tb_lab2_integrated.sv를 열고 해당 구간을 직접 입력한다.

```
65     check({screen[0],screen[1],screen[2],screen[3],screen[4]}=="MODE ", "LCD first line prefix");
66     check(screen[5]=="0" && screen[6]==(8'h31+mode_number), "LCD displayed mode number");
67     for(c=7;c<16;c=c+1) check(screen[c]==" ", "LCD first line padded");
68     for(c=0;c<16;c=c+1) check(screen[16+c]==names[mode_number][127-c*8 -: 8], "LCD full second line");
69 end endtask
70 initial begin $dumpfile("wave.vcd"); $dumpvars(0,tb_lab2_integrated); end
71 initial begin #1000000; $fatal(1,"watchdog"); end
72 initial begin
73     names[0]="UP DOWN COUNTER "; names[1]="CLOCK DIVIDER ";
74     names[2]="REGISTER PAIR "; names[3]="SHIFT REGISTER ";
75     names[4]="PISO "; names[5]="MOORE FSM ";
76     names[6]="MEALY FSM "; names[7]="8 DIGIT SCAN ";
77     for(i=0;i<32;i=i+1) screen[i]=0;
78     #2; rst=1; repeat(3) step; rst=0; repeat(4) step;
79     check(dut.mode==0 && led==0, "power reset to first mode");
80     switches(0); push step; check(led==1, "counter increments once");
```

tb_lab2_integrated.sv · 81–96행

sim/tb_lab2_integrated.sv를 열고 해당 구간을 직접 입력한다.

```
81      switches(1); push_step; check(led==0,"counter decrements");
82      push_step; check(led==15,"counter wraps down"); check_lcd(0);
83
84      next_mode(1); check_lcd(1);
85      pulses=0;
86      repeat(30) begin step; if(led[4]) pulses=pulses+1; end
87      check(pulses==3,"divider ticks three times in thirty clocks");
88
89      next_mode(2); switches(8'h01); push_step; check(led==8'h00,"register load");
90      switches(8'h33); push_step; check(led==8'h3a,"register old-value transfer");
91      push_step; check(led==8'h33,"register next transfer"); check_lcd(2);
92
93      next_mode(3); switches(8'h80); push_step; check(led==8'h08,"shift input 1");
94      switches(0); push_step; check(led==8'h04,"shift input 0"); check_lcd(3);
95
96      next_mode(4); switches(8'h01); push_step; check(led==8'h01,"PISO load");
```

tb_lab2_integrated.sv · 97-112행

sim/tb_lab2_integrated.sv를 열고 해당 구간을 직접 입력한다.

```
97      switches(8'h40); push_step; check(led===8'h40,"PISO first shift");
98      push_step; check(led===8'h81,"PISO second shift"); check_lcd(4);
99
100     next_mode(5); switches(8'h80); check(led===0,"Moore input cannot advance alone");
101     push_step; check(led===1,"Moore first transition");
102     push_step; check(led===2,"Moore second transition");
103     push_step; check(led===0,"Moore wraps"); check_lcd(5);
104
105     next_mode(6); switches(0); check(led===0,"Mealy input zero");
106     switches(8'h80); check(led===2,"Mealy changes without step");
107     push_step; check(led===5,"Mealy new state and output");
108     switches(0); check(led===4,"Mealy input zero preserves state"); check_lcd(6);
109
110     next_mode(7); switches(8'h90); check_lcd(7);
111     repeat(64) begin
112         step;
```

tb_lab2_integrated.sv · 113–128행

sim/tb_lab2_integrated.sv를 열고 해당 구간을 직접 입력한다.

```
113     if(seg_com==8'hff) blank_slots=blank_slots+1;
114     else begin
115         active_slots=active_slots+1;
116         check(seg_com===(8'hff ^ (8'h80>>led[2:0])), "scan COM polarity and index");
117         if(led[2:0]==0) check(seg_data===8'hf6, "scan DIP digit 9");
118     end
119 end
120 check(active_slots===32 && blank_slots===32, "scan blanking duty");
121 next_mode(0); check(led===0, "mode wrap resets previous counter value"); check_lcd(0);
122 // Simultaneous mode and step: reset has priority over a core update.
123 mode_button=1; step_button=1; repeat(12) step;
124 mode_button=0; step_button=0; repeat(12) step;
125 check(dut.mode===1 && dut.count===0, "simultaneous controls reset old core");
126 rst=1; repeat(4) step;
127 check(dut.mode===0 && led===0 && lcd_e===0 && seg_com===8'hff, "reset during operation");
128 $display("LAB2 INTEGRATED PASS modes=8 checks=%0d", checks);
```

tb_lab2_integrated.sv · 129–131행

sim/tb_lab2_integrated.sv를 열고 해당 구간을 직접 입력한다.

```
129          $finish;  
130      end  
131  endmodule
```

XDC를 직접 작성한다

1. constraints 폴더의 XDC를 열고 전체 내용을 입력한다.
2. get_ports의 이름과 비트 번호를 board top 선언과 대조한다.
3. 같은 핀을 서로 다른 포트에 중복 지정하지 않는다.
4. File → Save All. Icarus 기능 시뮬레이션은 XDC를 사용하지 않는다.

lab2_integrated.xdc · 1-16행

constraints/lab2_integrated.xdc를 열고 해당 구간을 직접 입력한다.

```
1  # Combo II-DLD S75, 1 kHz main clock. See evidence/board-pin-provenance.json.
2  set_property PACKAGE_PIN B6 [get_ports {clk}]
3  set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
4  set_property PACKAGE_PIN K4 [get_ports {rst}]
5  set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
6  set_property PACKAGE_PIN N8 [get_ports {mode_button}]
7  set_property IOSTANDARD LVCMOS33 [get_ports {mode_button}]
8  set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
9  set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
10 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
12 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
13 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
14 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
15 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
16 set_property PACKAGE_PIN T1 [get_ports {sw[4]}]
```

lab2_integrated.xdc • 17-32행

constraints/lab2_integrated.xdc를 열고 해당 구간을 직접 입력한다.

```
17 set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]]}
18 set_property PACKAGE_PIN U2 [get_ports {sw[5]]}
19 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]]}
20 set_property PACKAGE_PIN W3 [get_ports {sw[6]]}
21 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]]}
22 set_property PACKAGE_PIN Y1 [get_ports {sw[7]]}
23 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]]}
24 set_property PACKAGE_PIN N5 [get_ports {led[0]]}
25 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]]}
26 set_property PACKAGE_PIN M1 [get_ports {led[1]]}
27 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]]}
28 set_property PACKAGE_PIN M3 [get_ports {led[2]]}
29 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]]}
30 set_property PACKAGE_PIN M7 [get_ports {led[3]]}
31 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]]}
32 set_property PACKAGE_PIN N7 [get_ports {led[4]]}
```

lab2_integrated.xdc • 33–48행

constraints/lab2_integrated.xdc를 열고 해당 구간을 직접 입력한다.

```
33 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]]}
34 set_property PACKAGE_PIN M2 [get_ports {led[5]]}
35 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]]}
36 set_property PACKAGE_PIN M4 [get_ports {led[6]]}
37 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]]}
38 set_property PACKAGE_PIN L4 [get_ports {led[7]]}
39 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]]}
40 set_property PACKAGE_PIN N4 [get_ports {step_button}]
41 set_property IOSTANDARD LVCMOS33 [get_ports {step_button}]
42 set_property PACKAGE_PIN H2 [get_ports {seg_data[0]]}
43 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[0]]}
44 set_property PACKAGE_PIN J7 [get_ports {seg_data[1]]}
45 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[1]]}
46 set_property PACKAGE_PIN J3 [get_ports {seg_data[2]]}
47 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[2]]}
48 set_property PACKAGE_PIN J1 [get_ports {seg_data[3]]}
```

lab2_integrated.xdc · 49–64행

constraints/lab2_integrated.xdc를 열고 해당 구간을 직접 입력한다.

```
49 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[3]}]
50 set_property PACKAGE_PIN E4 [get_ports {seg_data[4]}]
51 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[4]}]
52 set_property PACKAGE_PIN E2 [get_ports {seg_data[5]}]
53 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[5]}]
54 set_property PACKAGE_PIN F5 [get_ports {seg_data[6]}]
55 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[6]}]
56 set_property PACKAGE_PIN F1 [get_ports {seg_data[7]}]
57 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[7]}]
58 set_property PACKAGE_PIN K5 [get_ports {seg_com[0]}]
59 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[0]}]
60 set_property PACKAGE_PIN K3 [get_ports {seg_com[1]}]
61 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[1]}]
62 set_property PACKAGE_PIN K1 [get_ports {seg_com[2]}]
63 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[2]}]
64 set_property PACKAGE_PIN L6 [get_ports {seg_com[3]}]
```

lab2_integrated.xdc • 65–80행

constraints/lab2_integrated.xdc를 열고 해당 구간을 직접 입력한다.

```
65 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[3]]}
66 set_property PACKAGE_PIN G3 [get_ports {seg_com[4]]}
67 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[4]]}
68 set_property PACKAGE_PIN G1 [get_ports {seg_com[5]]}
69 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[5]]}
70 set_property PACKAGE_PIN H6 [get_ports {seg_com[6]]}
71 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[6]]}
72 set_property PACKAGE_PIN H4 [get_ports {seg_com[7]]}
73 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[7]]}
74 set_property PACKAGE_PIN A4 [get_ports {lcd_data[0]]}
75 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data[0]]}
76 set_property PACKAGE_PIN B2 [get_ports {lcd_data[1]]}
77 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data[1]]}
78 set_property PACKAGE_PIN C3 [get_ports {lcd_data[2]]}
79 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data[2]]}
80 set_property PACKAGE_PIN D4 [get_ports {lcd_data[3]]}
```

lab2_integrated.xdc • 81–96행

constraints/lab2_integrated.xdc를 열고 해당 구간을 직접 입력한다.

```
81 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data[3]]}
82 set_property PACKAGE_PIN A2 [get_ports {lcd_data[4]]}
83 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data[4]]}
84 set_property PACKAGE_PIN C5 [get_ports {lcd_data[5]]}
85 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data[5]]}
86 set_property PACKAGE_PIN C1 [get_ports {lcd_data[6]]}
87 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data[6]]}
88 set_property PACKAGE_PIN D1 [get_ports {lcd_data[7]]}
89 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data[7]]}
90 set_property PACKAGE_PIN A6 [get_ports {lcd_e}]
91 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_e}]
92 set_property PACKAGE_PIN G6 [get_ports {lcd_rs}]
93 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_rs}]
94 set_property PACKAGE_PIN D6 [get_ports {lcd_rw}]
95 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_rw}]
96 create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
```

lab2_integrated.xdc · 97-97행

constraints/lab2_integrated.xdc를 열고 해당 구간을 직접 입력한다.

```
97  set_false_path -from [get_ports {rst mode_button step_button sw[*]}]
```

VS Code에서 먼저 시뮬레이션

1. File → Save All.
2. Terminal → Run Task... → 02 Simulate.
3. 첫 오류가 있으면 파일 경로·행 번호와 이전 행의 세미콜론부터 확인.
4. LAB2_PASS 또는 LAB2_INTEGRATED_PASS와 검사 수·종료 시각 확인.
5. Terminal → Run Task... → 03 Open waveform.

파형에서 확인할 신호

1. clk·rst·제어 입력·저장값·출력을 파형에 추가한다.
2. 상승 에지 직전 입력과 에지 뒤 상태를 비교한다.
3. 리셋 우선·enable=0 유지·마지막 검사 시각을 확인한다.
4. 출력만 본 화면과 전체 파형을 각각 저장한다.
5. 새 실행 로그와 파형인지 확인한다. 이전 PASS를 재사용하지 않는다.

정상 실행의 기준

```
LAB2_INTEGRATED_PASS modes=8 checks=2848
```

Icarus 종료 시각: 34326ns. 로그가 ps로 표시되면 1,000으로 나눠 ns와 비교한다.

이 수치는 교안의 입력 순서·TB·파라미터에 대한 기준이다.

검사 수만 보지 말고 예상 상태와 실제 파형이 일치하는 이유를 설명한다.

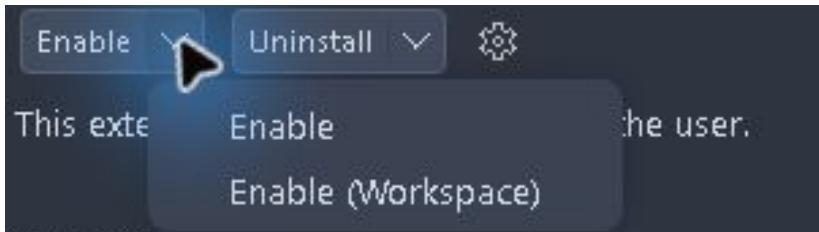
통합 자기검사 통과

```
LAB2_INTEGRATED_PASS modes=8 checks=2848
```

Ctrl+Shift+B 또는 Terminal → Run Task... → 02 Simulate.

LAB2_INTEGRATED_PASS modes=8 checks=2848을 확인한다. 종료 시각은 34326 ns다.

파형 확장 활성화



Extensions에서 @installed vaporview 검색. 꺼져 있으면 Enable 옆 화살표 → Enable (Workspace). 설치 전이면 앞의 추천 확장 절차를 따른다.

텍스트 대신 파형으로 열기



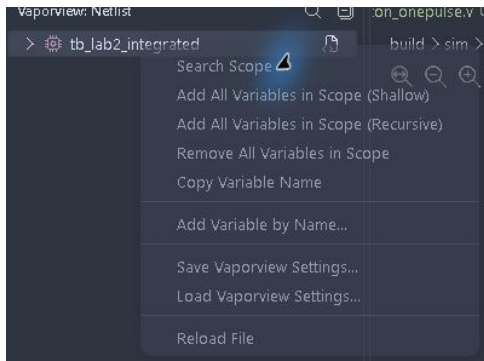
build/sim/wave.vcd를 연다. 텍스트로 열렀다면 편집기 오른쪽 위 Text Editor 옆 화살표 → VaporView를 선택한다.

Netlist View 열기



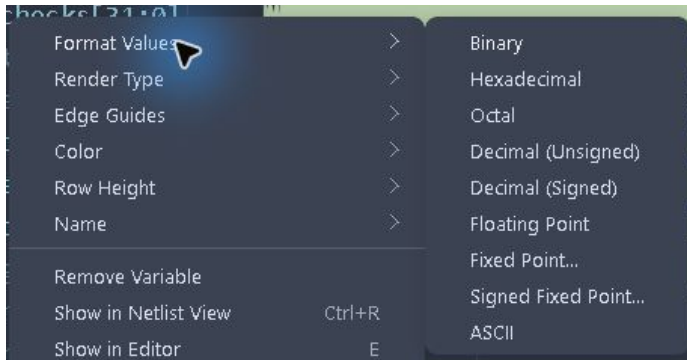
빈 파형 화면의 Netlist View를 누른다. 왼쪽에서 tb_lab2_integrated를 찾는다.

테스트벤치 신호와 내부 mode 추가



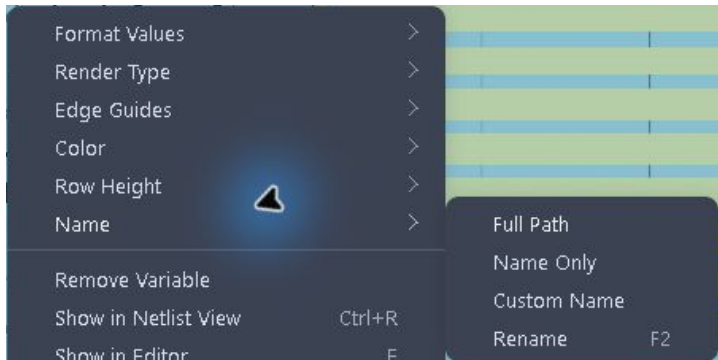
tb_lab2_integrated를 우클릭 → Add All Variables in Scope (Shallow).
이어서 tb_lab2_integrated → dut를 펼쳐 mode를 더블클릭한다. mode는
맨 아래에 추가된다.

검사 수는 십진수로 표시



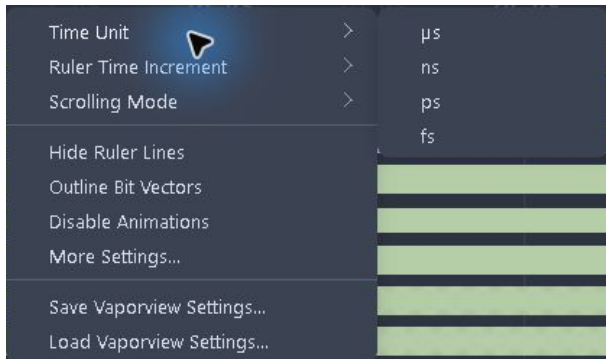
checks를 우클릭 → Format Values → Decimal (Unsigned). LCD 데이터와 LED·세그먼트 버스는 16진수로 둔다.

파형 이름을 읽기 쉽게



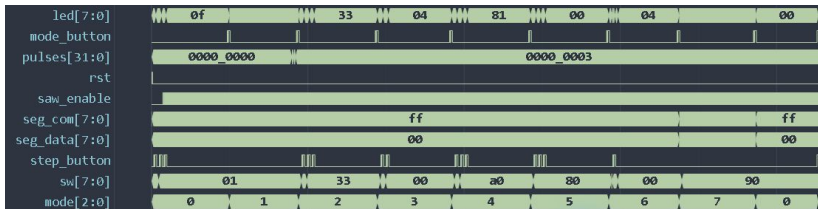
파형 영역에서 Ctrl+A → 신호 이름 우클릭 → Name → Name Only. 빈 영역을 클릭해 선택을 해제한다. Ctrl+B로 사이드바를 접으면 가로 공간이 넓어진다.

시간 단위와 전체 구간



시간 눈금을 우클릭 → Time Unit → μs . 왼쪽 위 첫 돋보기인 Zoom Fit을 누르면 전체 $34.326 \mu\text{s}$ 를 볼 수 있다.

8개 모드의 전환 확인



맨 아래 mode는 0~7을 순회한 뒤 0으로 돌아온다. mode_button과 step_button의 역할을 구분하고 led·sw·seg_com·seg_data를 함께 비교한다. mode=7에서만 7세그먼트 스캔이 활성화된다.

LCD에 MODE 05를 보내는 구간



Zoom In으로 약 17.1 μ s 부근을 확대한다. `lcd_data`의 80은 첫 줄 주소 명령(RS=0), 뒤의 4D 4F 44 45 20 30 35는 MODE 05 문자(RS=1)다. 내부 mode=4와 LCD 표시 번호 05를 연결해 설명한다.

LCD 데이터와 Enable의 관계

데이터가 바뀐 뒤 다음 클록에서 `lcd_e`가 올라가고, 그 다음 클록에서 내려간다. `E`가 높은 동안 데이터와 `RS`가 유지되는지 확대 파형에서 확인한다. `lcd_rw`는 항상 0이다.

테스트벤치는 10 ns 주기로 시간을 줄여 검사한다. 실제 보드의 1 kHz 클록에서는 한 클록이 1 ms이므로 파형의 ns 값을 장비의 동작 시간으로 그대로 쓰지 않는다.

LCD는 한 화면을 쓰기 시작할 때 모드를 저장한다. 버튼 입력 직후 화면 전체가 즉시 바뀌는 것으로 해석하지 말고 다음 화면 갱신을 확인한다.

수정 실험 · 실패와 복구

1. 정상 로그와 파형을 먼저 보관한다.
2. counter4.v의 증가량을 1에서 2로 바꾼다. TB 기대값은 유지한다.
3. Save All → 02 Simulate. LAB2_FAIL의 검사 이름·시각을 읽는다.
4. 어떤 입력과 이전 상태에서 불일치가 발생했는지 계산한다.
5. RTL을 복구·저장·재실행하고 전체 PASS를 확인한다.

실험 전 레포트

1. 동작 표·상태 전이·타이밍을 예상한다.
2. RTL·TB·XDC·설정의 역할을 설명한다.
3. 정상/변경/복구 3행 비교표와 실행 로그를 넣는다.
4. VS Code 파형을 확대해 입력·이전 상태·다음 상태를 설명한다.
5. 장비 클록·핀·예상 LED 동작을 정리한 뒤 Vivado/CLI로 진행한다.

통합 파형은 목적별로 나누어 본다

1. 제어 파형: `clk.rst.mode_button.step_button.dut.mode.dut.mode_press.dut.step_press.dut.circuit_reset`.
2. 회로 파형: `sw.led`와 현재 모드의 코어 저장값. 입력 변경, 버튼 펄스, 상태 갱신 순서를 확대한다.
3. LCD 파형: `lcd_e.lcd_rs.lcd_rw.lcd_data`. 번호와 이름을 전송하는 구간을 확인한다.
4. 스캔 파형: `seg_com.seg_data.dut.scan_index`. 선택 신호 FF가 blank 구간인지 확인한다.
5. 마지막 동시 버튼과 리셋 구간을 별도 화면으로 남기고 정상 로그와 함께 설명한다.

Vivado 없이 S75 구현

macOS에서는 해당 ARM64 배포, Windows에서는 WSL Linux 배포를 사용한다.

제작 검증은 WSL Linux x86-64에서 수행했다. Mac에서 실행했다는 뜻이 아니다.

합성·배치배선 도구는 OSS CAD Suite 2026.08.19와 openXC7 2026.08.20이다. 뒤의 절차로 고정 패키지를 직접 설치한다.

CLI 실행 환경 선택

Windows는 WSL Linux x86-64 터미널에서 실행한다. Apple Silicon Mac은 macOS 터미널과 darwin-arm64 패키지를 사용한다.

아래 명령은 같은 터미널에서 순서대로 실행한다. Python 3·Git·curl·tar를 먼저 확인한다.

```
python3 --version  
git --version  
curl --version  
tar --version  
uname -s -m
```

고정 릴리스와 설치 폴더

```
platform=linux-x86-64
tools="$HOME/fpga-cli/tools-lab2-202608"
mkdir -p "$tools/oss-cad-suite" "$tools/openxc7"
```

WSL x86-64는 위 platform을 쓴다. Apple Silicon Mac에서는 첫 줄을 platform=darwin-arm64로 바꾼다.

OSS CAD Suite 2026-08-19와 openXC7 2026-08-20을 직접 받는다. apio packages install은 원격의 최신 요구 버전을 따르므로 이 교안의 고정 버전 설치에 사용하지 않는다.

OSS CAD Suite 고정 패키지

```
base=https://github.com/fpgawars/tools-oss-cad-suite
base="$base/releases/download/2026-08-19"
file="apio-oss-cad-suite-$platform-20260819.tgz"
curl --fail --location "$base/$file" -o "$tools/oss.tgz"
tar -xzf "$tools/oss.tgz" -C "$tools/oss-cad-suite"
```

공식 OSS CAD Suite 고정 릴리스

다운로드가 실패하면 다음 줄을 실행하지 않는다. 압축 내부의 bin·lib 등은 지정한 설치 폴더 바로 아래에 풀린다.

openXC7 고정 패키지

```
base=https://github.com/fpgawars/tools-openxc7
base="$base/releases/download/2026-08-20"
file="apio-openxc7-$platform-20260820.tgz"
curl --fail --location "$base/$file" -o "$tools/openxc7.tgz"
tar -xzf "$tools/openxc7.tgz" -C "$tools/openxc7"
```

공식 openXC7 고정 릴리스

이 패키지는 nextpnr-xilinx·프레임 변환·bitread를 제공한다. 다른 FPGA의 part 데이터로 대체하지 않는다.

같은 터미널에서 도구 경로 설정

```
export PATH="$tools/oss-cad-suite/bin:$tools/openxc7/bin:$PATH"  
yosys -V  
nextpnr-xilinx --version  
iverilog -V  
vvp -V
```

검증 기록의 Yosys는 0.63+173 (66306a8ca), nextpnr는 68aeeb3이다.
다르면 PATH에서 어떤 실행 파일이 먼저 선택됐는지 확인한다.

새 터미널을 열면 tools 변수와 export PATH를 다시 실행한다. 다운로드.
압축 해제까지 매번 반복하지 않는다.

S75 준비 도구 받기

```
mkdir -p ~/fpga-cli/lab2-tools
cd ~/fpga-cli/lab2-tools
base=https://raw.githubusercontent.com/Claysia
base="$base/fpga-lab-example-opensource-cli-integrated"
base="$base/v2.0.1/tools"
curl --fail --location "$base/prepare_s75.py" \
-o prepare_s75.py
curl --fail --location \
"$base/xc7s75fgga484-1-package-pins.csv" \
-o xc7s75fgga484-1-package-pins.csv
```

정확한 S75 DB와 chipdb 준비

```
python3 prepare_s75.py --tools \  
~/fpga-cli/tools-lab2-202608/openxc7
```

준비 도구는 ~/fpga-cli/lab1-s75에 공통 장비 데이터를 만든다. LAB1과 같은 S75 DB이므로 디렉터리 이름을 유지한다.

prepared.json의 xc7s75fgga484-1.338핀·DB 고정 커밋·chipdb 해시를 확인한다.

실패하면 export.log·bbasm.log를 읽는다. 완료 후 자신의 lab2_integrated_cli 프로젝트 루트로 돌아간다.

synth.js 직접 작성

```
read_verilog src/*.v
synth_xilinx -family xc7 -top lab2_integrated
write_json build/cli/design.json
stat
```

TB를 합성 파일에 포함하지 않는다. 프로젝트 루트에 synth.js로 저장한다.

합성과 배치배선

```
mkdir -p build/cli
yosys -l build/cli/synthesis.log -s synth.py
chip="$HOME/fpga-cli/lab1-s75/xc7s75fpga484-1.bin"
db="$HOME/fpga-cli/lab1-s75/prjxray-db/spartan7"
part=xc7s75fpga484-1
nextpnr-xilinx --chipdb "$chip" \
--xdc constraints/lab2_integrated.xdc \
--json build/cli/design.json --fasm build/cli/design.fasm \
--freq 0.001 --log build/cli/place-route.log
```

프레임과 bit 생성

```
fasm2frames --db-root "$db" --part "$part" \  
build/cli/design.fasm build/cli/design.frames  
xc7frames2bit --part_file "$db/$part/part.yaml" \  
--part_name "$part" --frm_file build/cli/design.frames \  
--output_file build/cli/lab2_integrated.bit
```

각 단계 종료 코드와 로그를 확인한다. 이전 bit 파일을 새 실행 결과로 쓰지 않는다.

openFPGALoader 설치 · Ubuntu

WSL Ubuntu 터미널에서 실행한다. 아래 설치하는 Windows PowerShell 명령이 아니다.

```
sudo apt update  
sudo apt install openfpgaloader usbutils  
openFPGALoader --help  
openFPGALoader --list-cables
```

공식 운영체제별 설치 안내

openFPGALoader 설치 · macOS

Apple Silicon Mac은 macOS 터미널에서 아래 명령을 사용한다. Ubuntu 사용자는 이 장을 건너뛰는다.

```
brew install openfpgaloader
openFPGALoader --help
openFPGALoader --list-cables
```

Homebrew가 먼저 설치되어 있어야 한다. 이 교안의 실행 검증은 WSL Linux이며 macOS 실행 기록은 아니다.
공식 macOS 설치 안내

WSL 에 JTAG 전달 · Windows 준비

Windows PowerShell에서 실행한다. WSL 2 Ubuntu 터미널도 열어 둔다.
이미 usbipd-win 5 이상이 설치되어 있으면 설치 명령은 건너뛴다.

```
winget install --interactive --exact dorssel.usbipd-win  
usbipd --version  
usbipd list
```

목록에서 실습 보드의 USB JTAG 장치를 찾는다. 다음 장의 MY_BUSID를 그
행의 BUSID로 바꾼다.

Microsoft WSL USB 연결 안내

WSL에 JTAG 전달 · 연결

관리자 PowerShell에서 한 번 공유한 뒤, 일반 PowerShell에서 연결한다.
MY_BUSID에는 앞에서 확인한 BUSID를 넣는다.

```
usbipd bind --busid MY_BUSID  
usbipd attach --wsl --busid MY_BUSID  
usbipd list
```

bind만 관리자 권한이 필요하다. 연결 상태가 Attached인지 확인한다.
WSL에 연결한 동안 Windows 프로그램은 해당 USB 장치를 사용할 수 없다.

WSL에서 장치 확인과 반환

WSL Ubuntu 터미널에서 장치가 보이는지 먼저 확인한다.

```
lsusb  
openFPGALoader --list-cables  
openFPGALoader --detect
```

권한 오류는 공식 udev 설정 안내를 확인한다. FPGA 탐지가 실패하면 bit 기록으로 진행하지 않는다. 실습이 끝나면 Windows PowerShell에서 반환한다.

```
usbipd detach --busid MY_BUSID
```

공식 USB 접근 권한 안내

bit 읽기와 실제 기록

```
bitread --part_file "$db/$part/part.yaml" -C -z -y \  
-o build/cli/bits.txt build/cli/lab2_integrated.bit  
openFPGALoader --list-cables  
openFPGALoader --detect  
openFPGALoader -c MY_CABLE build/cli/lab2_integrated.bit
```

MY_CABLE은 실제 연결 케이블 식별자로 바꾼다. SRAM 기록이며 Flash 옵션 -f는 넣지 않는다.

CLI 검증 범위

현재 S75 bit 생성·IDCODE·9,104프레임 읽기 결과는 예시의 evidence/openxc7에 있다.

nextpnr는 set_false_path를 무시한다. 이 예외는 Vivado 쪽 비동기 입력 타이밍 분석에만 적용된다.

파일 검사 성공은 실제 버튼·LCD·장치 프로그래밍 성공과 구분한다.

장치에서 동작 확인

1. 보드 전원·JTAG·주 클록 1 kHz 설정을 확인한다.
2. 앞 장의 openFPGALoader -detect로 연결된 FPGA를 확인한다.
3. 실제 케이블 이름으로 MY_CABLE을 바꾸고 방금 생성한 bit를 SRAM에 기록한다.
4. 종료 코드와 기록 로그를 확인한 뒤 리셋 버튼을 누른다.
5. 모드 버튼으로 8개 모드를 순회하며 LCD 이름과 회로 출력을 예상과 비교한다.

통합판 영상의 촬영 순서

1. K4 초기화 후 LCD MODE 01과 카운터 동작을 촬영한다.
2. N8을 누르고 놓아 다음 이름으로 바뀌는 장면을 찍는다. 회로 한 단계는 N4로 실행한다.
3. 8개 모드마다 LCD 이름·스위치 입력·출력을 함께 보여 준다.
4. MODE 08의 여덟 자리와 MODE 01로 돌아갈 때 카운터 초기화를 보여 준다.
5. 실제로 기록한 bit의 경로·해시와 사진·영상 링크를 README 및 실험 후 레포트에 연결한다.

실험 후 레포트와 GitHub

1. 도구 버전·source commit·XDC·bit 해시·타이밍과 경고를 적는다.
2. 실제 기록 화면과 보드 전체 사진을 포함한다.
3. 버튼 누름과 출력 변화가 함께 보이는 영상을 찍는다.
4. 통합판은 8개 모드 번호·LCD 이름·회로 출력을 보여 준다.
5. 소스·실험 전후 레포트·사진·영상 링크를 GitHub에서 연결한다.